

Document number: P1471r0
Date: 2019-01-20
Project: Programming Language C++
Audience: EWG
Reply-to: Christopher Kohlhoff <chris@kohlhoff.com>

The trouble with `coroutine_traits`

Introduction

This paper describes some issues encountered during the use of the Coroutines TS [N4775](#) in some real libraries and applications. These issues relate to `coroutine_traits` specifically, and in particular how `coroutine_traits`:

- couples implementation to interface; and
- is a global registry.

Issues

Implementation is coupled to interface

When presented with a function declaration of the form:

```
std::future<int> foo(Arg1, Arg2);
```

a user does not know whether the implementation is a coroutine or not. This is a good thing, as it separates interface from implementation. In this case the use of a coroutine is an implementation detail.

The problem is that if `foo` is implemented as a coroutine then it must be a particular implementation as determined by `coroutine_traits<future<int>, Arg1, Arg2>::promise_type`.

Cannot specialize `coroutine_traits` for standard types

The first problem is that we are not allowed to partially specialize `coroutine_traits` using only standard types.

To work around this without changing our library interface, we instead implement our coroutine inside a lambda, with a tag argument:

```
std::future<int> foo(Arg1, Arg2)
{
    return [](my_tag_type, Arg1, Arg2)
    {
        // ...
    }();
}
```

We then partially specialize `coroutine_traits`

```
template<class R, class... Args>
struct coroutine_traits<future<R>, my_tag_type, Args...>;
```

(An alternative approach is to include the tag type in our `foo` function signature. However, this leaks implementation details into the interface which is exactly what we are trying to avoid.)

coroutine_traits represents a global registry of behaviour

For the sake of argument, let's assume that the standard library already specializes `coroutine_traits` for `std::future` return types. This specialization's promise type might exhibit a behaviour that is unacceptable for our use case (for example, we might want to enforce that `await_ready` always return false for any `co_await` operations it performs).

As `coroutine_traits` is effectively a global registry of coroutine promise types, we must once again employ our tag-based coroutine lambda to select an alternative implementation.

Cannot specialize coroutine_traits for arbitrary (potentially builtin) types

`coroutine_traits` aside, the Coroutines TS syntax *can* be used in the implementation of functions that have arbitrary return types and arguments:

```
template<class T, class U>
auto something_generic(T t, U u)
{
    // ...
}
```

Examples of when we want to do this include:

- employing coroutines for lightweight error handling; and
- writing algorithms that can be used in both coroutine and non-coroutine contexts

Once again, we must use our tag-based coroutine lambda workaround.

Lambdas, lambdas everywhere

As a consequence of the above issues we find that, over time, function implementations employing coroutines-as-lambdas tend to proliferate. For those concerned about the teachability of coroutines (and in particular the concern that the coroutine lambda syntax of [P1063](#) was less teachable), this represents a leap in complexity for commonly encountered use cases.

Danger of conflicting coroutine_traits specializations

Another consequence of this tag-based approach to specialization is that if the standard later introduces its own partial specializations:

```
template<class R, class... Args> struct coroutine_traits<future<R>, Args...>;
template<class... Args> struct coroutine_traits<future<void>, Args...>;
```

our partial specializations are now ambiguous. A similar problem of ambiguity can occur between unrelated third-party libraries.

Proposed solution

This paper proposes to eliminate `coroutine_traits` and instead employ a syntax similar to:

```
R f(A1, A2, ..., An) coroutine<C>
{
}
```

Whether or not a given function is a coroutine is an implementation detail of that function. Thus, the trailing `coroutine<C>` annotation would be applied to a function definition, rather than its declaration. It belongs with the definition to reflect the fact that it is an implementation detail.

`C` is a name that is used by the compiler in the expression `C<R, A1, A2, ... An>` to determine the coroutine's promise type.

For example:

```
// Declaration of coroutine function:
future<int> foo(Arg1, Arg2);

// Alias template used to determine coroutine promise type:
template <class R, class... Args>
using task = /* ... */;

// Definition of coroutine function:
future<int> foo(Arg1 a1, Arg2 a2) coroutine<task>
{
    // ...
}
```

This approach eliminates the leap in complexity when using lambdas to address these use cases.

An added bonus

One useful by-product of an explicit annotation is that we no longer require the `co_return` keyword. The remaining keywords `co_await` and `co_yield` are now only valid in this explicitly annotated context, so it may also be feasible to drop the `co_` prefix.

References

[N4775] G. Nishanov. *Working Draft, C++ Extensions for Coroutines*.

[P1063] G. Romer, J. Dennett, C. Carruth. *Core Coroutines*.